

slug / FLM ASR Latency Investigation

Session record
Project: rahlquist/slug-tts-stt
Scope: user request beginning “In the process of setting up slug you asserted FLM has...” through restoration and update review.
Hosts: slug.home.lan (Krackan voice edge), Hermes execution host, upstream FastFlowLM source.
Status: investigation complete; production stack restored; no production binary or persistent unit changed by instrumentation.

1. User request and investigation objectives

The user asked whether the previously reported approximately 1.6–2.2 second FLM ASR delay could be narrowed down, specifically requesting:

- Determine how long the 3,000 mel frames take to compute.
- Thoroughly inspect audio_chunk.
- Consult source code, documentation, bug tickets, and upstream material.
- Build and run an instrumented FLM build if possible.
- Perform orderly tests to locate the slowdown.
- Restore FLM to its pre-test state, with instrumentation removed or disabled.
- Check current FLM, ROCm, AMD/XDNA/XRT updates afterward.
- Document all test stages and code alterations so results are auditable.

2. Baseline inherited from the earlier slug deployment

The existing Option B architecture was:

STT: client → slug:8081 whisper-proxy → slug:8090 FLM Whisper on XDNA2 NPU
TTS: client → slug:8092 qwentts.cpp Qwen3-TTS on Radeon 840M iGPU

Earlier measured values were:

Test	Result
0.5 s audio	approximately 2.16–2.3 s
2.5 s audio	approximately 2.32–2.4 s
11 s audio	approximately 2.7–2.8 s
Direct FLM vs proxy	effectively identical
Strict simultaneous STT + TTS	STT +1.8%; TTS +3.3%; TTFA +0.8%
End-to-end target	3.73 s vs working target <1 s

The earlier hypothesis was “fixed overhead inside FLM.” This investigation was designed to split that overhead into audio loading, mel preprocessing, NPU encoding, and token decoding.

3. Phase 1 — environment and production-state inspection

No production change was made initially. The live service was inspected:

```
FLM v0.9.46
ExecStart=/usr/bin/flm serve whisper-v3:turbo --asr 1 --host 127.0.0.1 --port 8090
PID resident for more than 18 hours
RSS approximately 7.3 GB
/dev/accel/accel0 open
LimitMEMLOCK=infinity
```

The service defaulted to --pmode performance; --pmode turbo was not present in the persistent unit.

FLM help exposed the relevant controls: --pmode, --ctx-len, --prefill-chunk-len, --q-len, and --asr.

4. Phase 2 — tighter direct/proxy latency reproduction

A repeatable probe ran five requests per endpoint using the same files and measured wall time with `curl`.

Audio	Direct :8090	Proxy :8081	Conclusion
0.5 s	mean 2.2986 s	mean 2.2434 s	no material proxy cost
2.5 s	mean 2.3693 s	mean 2.3523 s	no material proxy cost
11 s	mean 2.7747 s	mean 2.7550 s	no material proxy cost

All requests returned HTTP 200. Repeated requests stayed flat, ruling out per-request model reload.

5. Phase 3 — source review

5.1 Fixed 30-second Whisper window

Current upstream source was cloned from R0Cm/FastFlowLM at commit:

```
8279cd6786bdd0b4f1ee8e9107c4a436322ce40b
Merge pull request #658 from R0Cm/repo-transfer
```

In `src/common/whisper/modeling_whisper_audio.cpp`:

```
const int CHUNK_LENGTH = 30;
const int N_SAMPLES = SAMPLE_RATE * CHUNK_LENGTH; // 480000

if (x.size() < N_SAMPLES)
    x.resize(N_SAMPLES, 0.0f);

int n_frames = (...) / HOP_LENGTH + 1; // 3000
```

The mel path therefore computes a fixed 128×3000 feature tensor for short input. In the source, the work is clearly separate from `encode_audio()`.

5.2 audio_chunk construction bug

The original code was:

```
std::vector<float> audio_chunk(l_this_round);
audio_chunk.insert(audio_chunk.begin(),
    this->audio_buffer.data() + current_idx,
    this->audio_buffer.data() + current_idx + l_this_round);
```

This constructs `l_this_round` zeroes and then inserts the real audio before them. Measured vector sizes:

Input	Real samples	Vector after original insert
0.5 s	8,000	16,000
2.5 s	40,000	80,000
11 s	176,000	352,000

This can cause incorrect truncation for longer inputs when preprocessing limits the vector to the 30-second window. It is a real correctness bug, but testing the corrected allocation showed it is not the 2.2-second latency cause.

5.3 Source audio loading

The source decodes the request from memory with FFmpeg, resamples to 16 kHz, converts to mono, and normalizes to float. That path was separately timed.

6. Phase 4 — 30-second boundary test

Temporary 29.5-second and 30.5-second WAVs were generated and sent directly to FLM:

Input	Measured latency
29.5 s	2.23–2.43 s
30.5 s	7.29–7.32 s

This sharp step confirms the sliding/multiple-window path. It is consistent with a fixed 30-second encoder invocation per window.

7. Phase 5 — instrumented FLM build

7.1 Build process

- Cloned current upstream FastFlowLM source and initialized submodules.
- Configured a separate CMake build under /tmp/FastFlowLM/src/build-instrumented.
- Used CMake, GCC, FFmpeg, FFTW, Boost, XRT, and AMD XDNA runtime already present on slug.
- The current tokenizer submodule required Rust/Cargo; Rust was installed user-local only, used for the build, then removed.
- The first parallel build hit a tokenizer generation race. A serial make -j1 completed successfully.

7.2 Exact instrumentation

Temporary edits were made only to src/common/whisper/modeling_whisper.cpp:

- Added <chrono> and a monotonic millisecond helper.
- Timed vector audio loading/resampling.
- Printed chunk source size and post-construction size with prefix [FLM-PROBE].
- Timed _preprocess_audio().
- Timed whisper_engine->encode_audio().
- Timed decoder execution.

The production /usr/bin/flm was not edited or replaced.

7.3 Instrumented results

Stage	0.5 s	2.5 s	11 s
Audio load/decode/resample	1.3 ms	4.3 ms	4.3 ms
Mel preprocessing, all 3,000 frames	5.5 ms	6.4 ms	6.4 ms
NPU encode_audio()	2365.8 ms	2186.2 ms	2183.2 ms
Token decode	100.6 ms	218.5 ms	613.1 ms
HTTP total	2474.1 ms	2416.6 ms	2809.0 ms

Finding: the fixed latency floor is the NPU Whisper encoder processing the fixed 30-second acoustic tensor. The 3,000 mel frames take approximately 6 ms, not 2 seconds.

8. Phase 6 — corrected audio_chunk experiment

The instrumented source was changed temporarily to:

```
std::vector<float> audio_chunk(  
    this->audio_buffer.begin() + current_idx,  
    this->audio_buffer.begin() + current_idx + 1_this_round);
```

The corrected vector sizes became 8,000 / 40,000 / 176,000 samples. Encoder timings remained approximately 2.15–2.38 seconds. End-to-end timing did not materially improve. This separates the correctness bug from the performance bottleneck.

9. Phase 7 — power-mode experiment

The temporary instrumented binary was run with -pmode turbo. A 0.5-second request measured approximately 2.37 seconds of encoder time, statistically indistinguishable from performance mode at approximately 2.38 seconds. Turbo mode is not the lever.

10. Upstream documentation and issue review

Reviewed:

- FastFlowLM Whisper documentation: docs/models/whisper.md [https://github.com/ROCm/FastFlowLM/blob/main/docs/docs/models/whisper.md]
- FastFlowLM issue #637: ASR response_format/timestamps [https://github.com/ROCm/FastFlowLM/issues/637]
- Current ROCm/FastFlowLM source and CMake build files.

Issue #637 confirms the ASR engine internally performs timestamp-aware sliding-window work, while the REST layer currently returns only minimal text. No upstream documentation found publishes a latency baseline for this path. Karen's independent review agreed that the full-window encoder path is the primary hypothesis and that the chunk construction is a separate bug.

11. Production restoration and verification

After testing:

- Stopped the temporary instrumented process.
- Removed temporary source, build, logs, test files, and user-local Rust/Cargo installation.
- Started the original systemd units: flm-asr, qwen-tts, whisper-proxy.
- Confirmed the production binary hash remained:

```
8f5a919fc39d10d71135baac863ae2eee5cd526ddd01e64e6c504a0392e970f4 /usr/bin/flm
```

Verification	Result
flm-asr	active, enabled
qwen-tts	active, enabled
whisper-proxy	active, enabled
Ports 8081/8090/8092	listening
Voices	trimmed and freeman restored
flm validate	clean; memlock infinity; NPU firmware 1.1.2.64; amdxdna 0.10
Direct and proxy STT	HTTP 200 and transcripts returned

12. Update review

Component	Installed state
FastFlowLM	0.9.46-1.1
XRT	2.21.75-10.1
XRT AMD XDNA plugin	2.21.75-2.1
ROCm	7.2.4
Mesa	26.1.6
Kernel	7.2.0-rc6-1-cachyos-rc
NPU firmware	1.1.2.64
amdxdna	0.10

FastFlowLM has moved from the original repository to the AMD-owned ROCm/FastFlowLM repository. Newer AMD Ryzen AI documentation advertises newer Ubuntu-oriented XRT/XDNA packages, but they were not installed because they are not verified drop-in replacements for this Arch/CachyOS stack. No update was applied.

13. Final findings

1. Mel computation for 3,000 frames is approximately **6 ms**.

2. The fixed ~2.2-second cost is the NPU Whisper encoder.
3. Short input is padded to a fixed 30-second, 3,000-frame representation.
4. The 30-second boundary produces a large multi-window latency jump.
5. The original `audio_chunk` construction doubles the vector and can lose tails on longer input; it should be fixed upstream.
6. Fixing `audio_chunk` does not remove the encoder latency.
7. Proxy, container format, resampling, model reload, and turbo power mode are not the dominant causes.
8. The most promising optimizations are dynamic/streaming encoder handling, incremental state, or a runtime/model path that avoids a full fixed 30-second encoder invocation.

14. Provenance and reproducibility

- **Measured:** all latency tables, stage timings, vector sizes, service hashes/states, endpoint results.
- **Source-derived:** 30-second padding, 3,000-frame mel tensor, encoder call order, chunk construction semantics.
- **Upstream evidence:** FastFlowLM source, Whisper documentation, issue #637, current repository commit listed above.
- **Not claimed:** no unverified “fix” was installed; no production binary was replaced; no vendor timing claim was treated as a measurement.

Generated as a sanitized project record. Credentials, tokens, secret-store identifiers, and private authentication material are intentionally excluded.